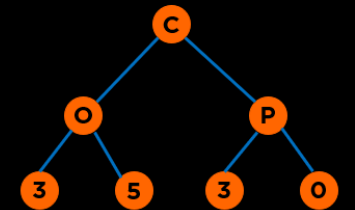
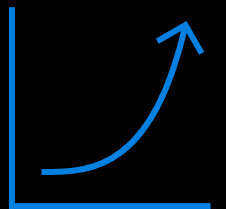


# Algorithm Analysis



# Agenda



- What is an Algorithm?
- Difference between a Program and an Algorithm
- Multiple Ways of Solving a Problem
- Benefits of Evaluating an Algorithm
- **How can we evaluate programs?**
  - Approach 1 (Simulation: Timing)
  - Approach 2 (Modeling: Counting)
  - Approach 3 (Asymptotic Behavior: Order of Growth)

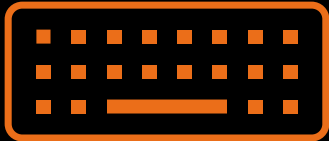
# Algorithm

# Algorithm

**An algorithm is a step-by-step procedure for solving a problem.**

# Algorithm

An algorithm is a step-by-step procedure for solving a problem.



**Input**



**Output**



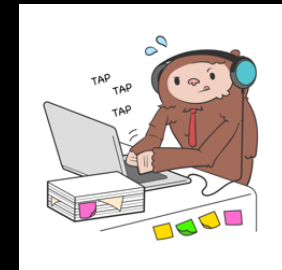
**Definite &  
Unambiguous**

# Algorithm vs Program

|                                | Algorithm | Code or Program |
|--------------------------------|-----------|-----------------|
| Focus of a professional        |           |                 |
| Form                           |           |                 |
| Dependence on H/W or OS        |           |                 |
| Professional's Cognitive State |           |                 |
| Correctness/Performance        |           |                 |

# Algorithm vs Program

|                                | Algorithm  | Code or Program      |
|--------------------------------|------------|----------------------|
| Focus of a professional        | Design     | Implementation       |
| Form                           | Pseudocode | Programming Language |
| Dependence on H/W or OS        | No         | Yes                  |
| Professional's Cognitive State | Thinking   | Doing                |
| Correctness/Performance        | Analysis   | Testing              |



# Multiple Ways of Solving a Problem

# Multiple Ways of Solving a Problem

**Problem:** Determine if a sorted array contains any duplicates.

|     |     |    |    |    |    |     |     |
|-----|-----|----|----|----|----|-----|-----|
| -13 | -11 | 12 | 14 | 24 | 24 | 100 | 102 |
|-----|-----|----|----|----|----|-----|-----|

# Multiple Ways of Solving a Problem

**Problem:** Determine if a sorted array contains any duplicates.

|     |     |    |    |    |    |     |     |
|-----|-----|----|----|----|----|-----|-----|
| -13 | -11 | 12 | 14 | 24 | 24 | 100 | 102 |
|-----|-----|----|----|----|----|-----|-----|

**Silly algorithm:** Every possible pair

# Multiple Ways of Solving a Problem

**Problem:** Determine if a sorted array contains any duplicates.

|     |     |    |    |    |    |     |     |
|-----|-----|----|----|----|----|-----|-----|
| -13 | -11 | 12 | 14 | 24 | 24 | 100 | 102 |
|-----|-----|----|----|----|----|-----|-----|

**Silly algorithm:** Every possible pair

**Better algorithm:** Compare adjacents

<https://onlinegdb.com/SHYJzMAxD>

# Multiple Ways of Solving a Problem

**Problem:** Determine if a sorted array contains any duplicates.

|     |     |    |    |    |    |     |     |
|-----|-----|----|----|----|----|-----|-----|
| -13 | -11 | 12 | 14 | 24 | 24 | 100 | 102 |
|-----|-----|----|----|----|----|-----|-----|

**Silly algorithm:** Every possible pair

**Better algorithm:** Compare adjacents

**Now that we know, that there are multiple ways to solve a problem,  
how do we evaluate which one is better?**

**Are all  
programs/algorithms  
equal in terms of  
performance?**

# Performance

In terms of what?

Are all programs/algorithms equal in terms of **performance**?

# Performance

## In terms of what?

- Time
- Space
- Energy Consumption
- Error Rates (Approximation Algorithms)

**Are all programs/algorithms equal in terms of performance?**

# Why do we care about algorithms?

# Why do we care about algorithms?

- **Knowing**



- **Experiencing**



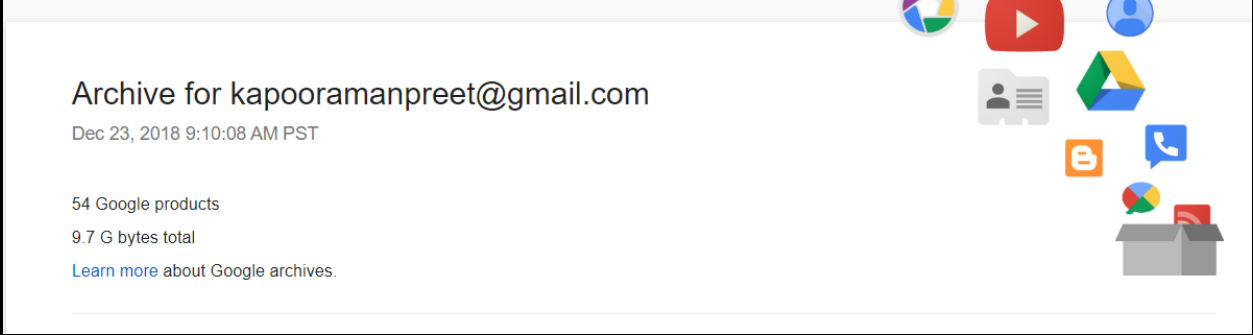
- **Selling**



- **Cost**

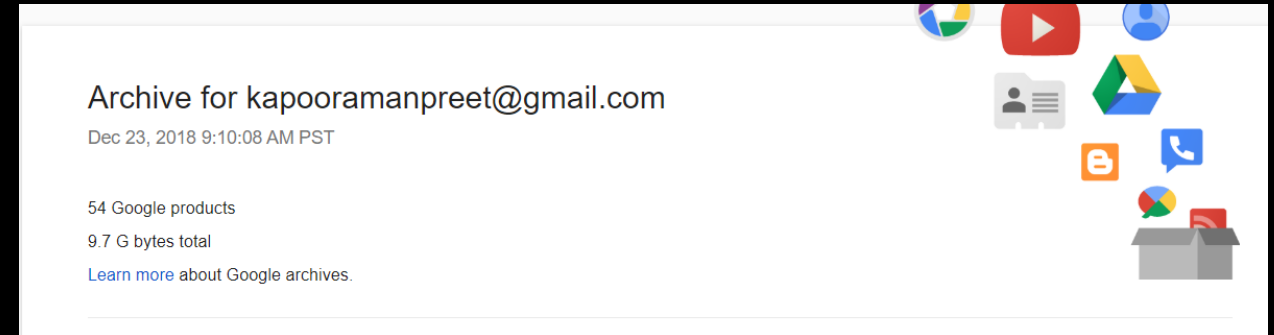


# A Simple Example



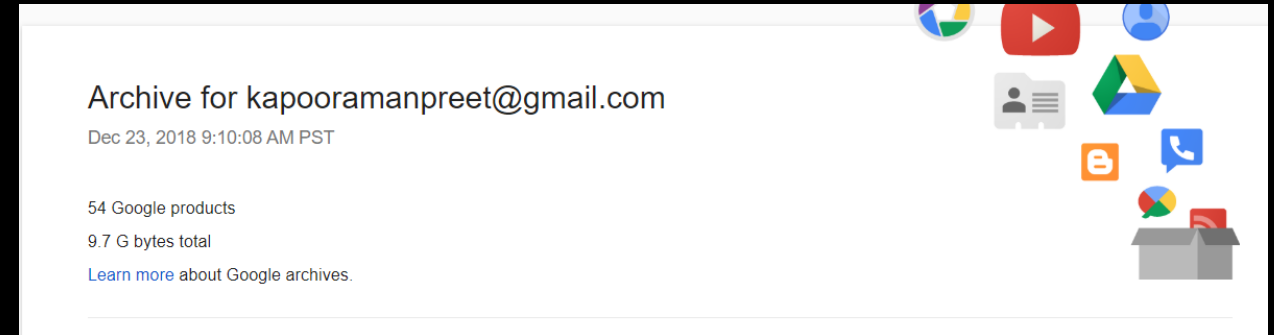
# A Simple Example

- Google has 2 billion active users
- Conservative estimate ~ 1 GB of data/user
- Total Data (Space):



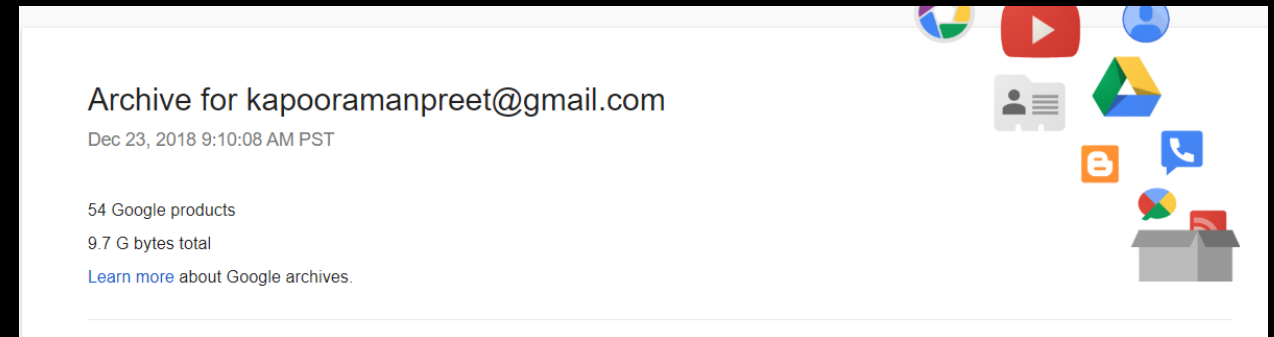
# A Simple Example

- Google has 2 billion active users
- Conservative estimate ~ 1 GB of data/user
- Total Data (Space): 2 Exabytes
- Total Time:



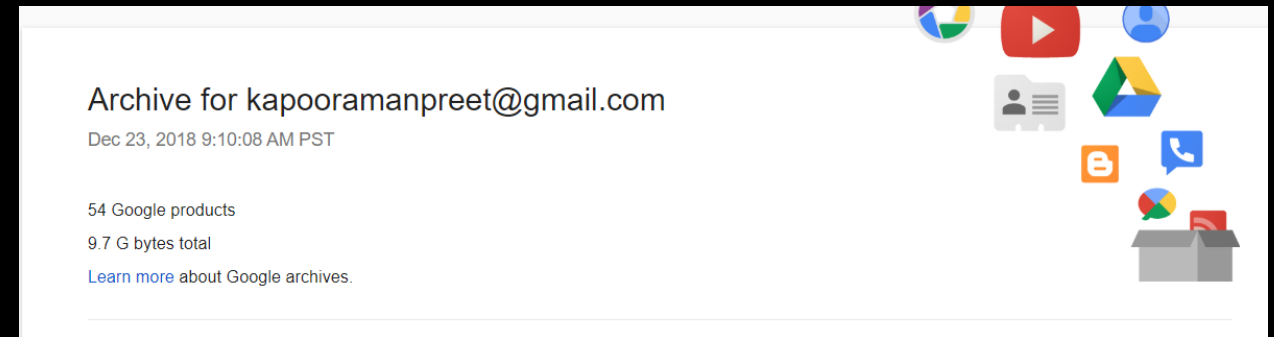
# A Simple Example

- Google has 2 billion active users
- Conservative estimate ~ 1 GB of data/user
- Total Data (Space): 2 Exabytes
- Total Time:
  - Operation Speed: 0.5 ns
  - Linear Search
  - Binary Search



# A Simple Example

- Google has 2 billion active users
- Conservative estimate ~ 1 GB of data/user
- Total Data (Space): 2 Exabytes
- Total Time:
  - Operation Speed: 0.5 ns
  - Linear Search: 11574 days or 31 years
  - Binary Search: 31s



**In short, we care about  
performance ...**

**So, how do we  
measure performance?**

# Questions to ask when evaluating programs

- **Time:** How much time does this take?
- **Space:** How much space does this consume?
- **Data:** Are there any patterns in our data?

# Approach 1 (Simulation: Timing)



# Approach 1 (Simulation: Timing)

## Code #1

```
01 auto t1 = Clock::now();  
02 for(int i=0; i<1000; i++);  
03 auto t2 = Clock::now();  
04 Print t2-t1
```

## Code #2

```
01 auto t1 = Clock::now();  
02 for(int i=0; i<1000000; i++);  
03 auto t2 = Clock::now();  
04 Print t2-t1
```

# Approach 1 (Simulation: Timing)

## Code #1

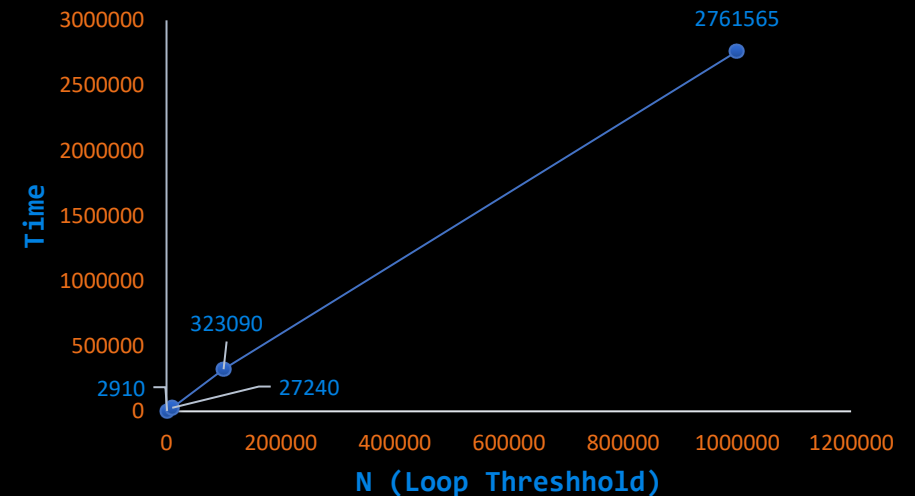
```
01 auto t1 = Clock::now();  
02 for(int i=0; i<1000; i++);  
03 auto t2 = Clock::now();  
04 Print t2-t1
```

## Code #2

```
01 auto t1 = Clock::now();  
02 for(int i=0; i<1000000; i++);  
03 auto t2 = Clock::now();  
04 Print t2-t1
```

## Output

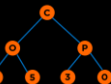
```
Delta t = t2-t1 (1000): 2910 nanoseconds  
Delta t = t2-t1 (10000): 27240 nanoseconds  
Delta t = t2-t1 (100000): 323090 nanoseconds  
Delta t = t2-t1 (1000000): 2761565 nanoseconds
```



<https://onlinegdb.com/BJGAP7I5I>

# Approach 1 (Simulation: Timing)

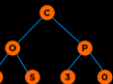
| Pros | Cons |
|------|------|
|      |      |
|      |      |
|      |      |
|      |      |
|      |      |



# Approach 1 (Simulation: Timing)

| Pros              | Cons                                         |
|-------------------|----------------------------------------------|
| Easy to measure   | Results vary across machines                 |
| Easy to interpret | Compiler dependent                           |
|                   | Results vary across implementations          |
|                   | Not predictable for small inputs             |
|                   | No clear relationship between input and time |

# Approach 2 (Modeling: Counting)



# Approach 2 (Modeling: Counting)

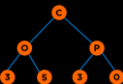
Count the number of operations

# Approach 2 (Modeling: Counting)

## Count the number of operations

|    |                        |
|----|------------------------|
| 01 | int sum=0;             |
| 02 | for(int i=0; i<n; i++) |
| 03 | sum += i;              |
| 04 | print sum              |

| Operation  | Symbolic count |
|------------|----------------|
| int sum=0; |                |
| int i=0;   |                |
| i<n;       |                |
| i++        |                |
| sum += i;  |                |
| print sum  |                |
| $T(n)$     |                |

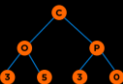


# Approach 2 (Modeling: Counting)

## Count the number of operations

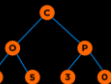
|    |                        |
|----|------------------------|
| 01 | int sum=0;             |
| 02 | for(int i=0; i<n; i++) |
| 03 | sum += i;              |
| 04 | print sum              |

| Operation   | Symbolic count    |
|-------------|-------------------|
| int sum=0;  | 1                 |
| int i=0;    | 1                 |
| i<n;        | $0 \dots n = n+1$ |
| i++         | n                 |
| sum += i;   | n                 |
| print sum   | 1                 |
| <b>T(n)</b> | <b>3n+4</b>       |



# Approach 2 (Modeling: Counting)

| Pros | Cons |
|------|------|
|      |      |
|      |      |
|      |      |
|      |      |



# Approach 2 (Modeling: Counting)

| Pros                                            | Cons                                |
|-------------------------------------------------|-------------------------------------|
| Independent of computer                         | All operations are equal            |
| Input dependence is captured in model (Scaling) | Tedious to compute                  |
|                                                 | Results vary across implementations |
|                                                 | Doesn't tell you actual time        |

# Approach 2 (Modeling: Counting)

## Count the number of operations

```
01 int sum=0;
02 for(int i=0; i<n; i++)
03     sum += i;
04 print sum
```

**Tedious to compute:**

**Different variables, so many operations, so many equations!**



| Operation                | Symbolic count           |
|--------------------------|--------------------------|
| <code>int sum=0;</code>  | 1                        |
| <code>int i=0;</code>    | 1                        |
| <code>i&lt;n;</code>     | $0 \dots n = n+1$        |
| <code>i++</code>         | $n$                      |
| <code>sum += i;</code>   | $n$                      |
| <code>print sum</code>   | 1                        |
| <b><math>T(n)</math></b> | <b><math>3n+4</math></b> |

# Approach 2 (Modeling: Counting)

## Count the number of operations

```
01 int sum=0;
02 for(int i=0; i<n; i++)
03     sum += i;
04 print sum
```

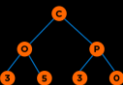
| Operation                | Symbolic count           |
|--------------------------|--------------------------|
| <code>int sum=0;</code>  | 1                        |
| <code>int i=0;</code>    | 1                        |
| <code>i&lt;n;</code>     | $0 \dots n = n+1$        |
| <code>i++</code>         | $n$                      |
| <code>sum += i;</code>   | $n$                      |
| <code>print sum</code>   | 1                        |
| <b><math>T(n)</math></b> | <b><math>3n+4</math></b> |

**Tedious to compute:**

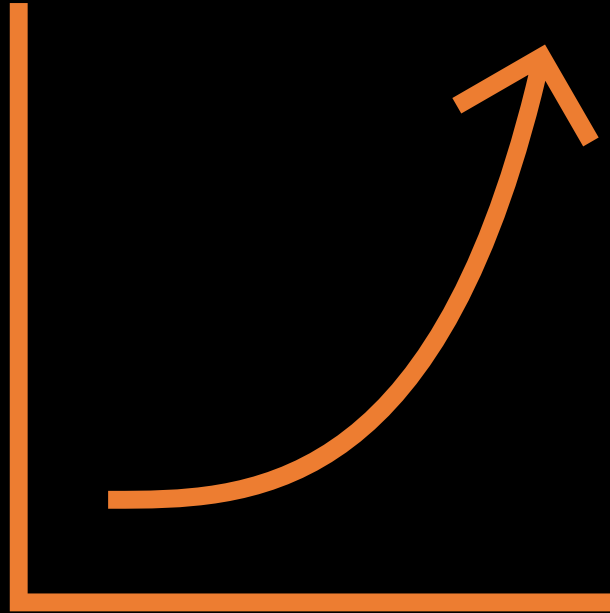
**Different variables, so many operations, so many equations!**



**Can we eliminate the complexity or get rid of extraneous variables?**



# Approach 3 (Asymptotic Behavior: Order of Growth)



# Which variables should we eliminate?

| Operation                | Symbolic count           |
|--------------------------|--------------------------|
| <code>int sum=0;</code>  | 1                        |
| <code>int i=0;</code>    | 1                        |
| <code>i&lt;n;</code>     | $0 \dots n = n+1$        |
| <code>i++</code>         | $n$                      |
| <code>sum += i;</code>   | $n$                      |
| <code>print sum</code>   | 1                        |
| <b><math>T(n)</math></b> | <b><math>3n+4</math></b> |

# Growth of Functions

Time,  $y = T(n)$

Inputs:  $n$

|           | 1 | $\log n$ | $n$ | $n \log n$ | $n^2$ | $n^3$ | $2^n$ | $n!$ |
|-----------|---|----------|-----|------------|-------|-------|-------|------|
| 1         |   |          |     |            |       |       |       |      |
| 10        |   |          |     |            |       |       |       |      |
| 100       |   |          |     |            |       |       |       |      |
| 1000      |   |          |     |            |       |       |       |      |
| 10000     |   |          |     |            |       |       |       |      |
| 100000    |   |          |     |            |       |       |       |      |
| 1000000   |   |          |     |            |       |       |       |      |
| 10000000  |   |          |     |            |       |       |       |      |
| 100000000 |   |          |     |            |       |       |       |      |

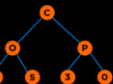
# Growth of Functions

Time,  $y = T(n)$

| Inputs: $n$ |           | 1 | $\log n$ | $n$ | $n \log n$ | $n^2$ | $n^3$ | $2^n$ | $n!$ |
|-------------|-----------|---|----------|-----|------------|-------|-------|-------|------|
|             | 1         | 1 |          |     |            |       |       |       |      |
|             | 10        | 1 |          |     |            |       |       |       |      |
|             | 100       | 1 |          |     |            |       |       |       |      |
|             | 1000      | 1 |          |     |            |       |       |       |      |
|             | 10000     | 1 |          |     |            |       |       |       |      |
|             | 100000    | 1 |          |     |            |       |       |       |      |
|             | 1000000   | 1 |          |     |            |       |       |       |      |
|             | 10000000  | 1 |          |     |            |       |       |       |      |
|             | 100000000 | 1 |          |     |            |       |       |       |      |

# Growth of Functions

## Time, $y = T(n)$



# Growth of Functions

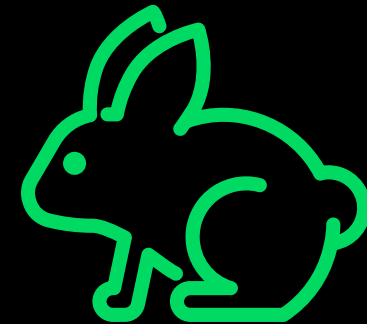
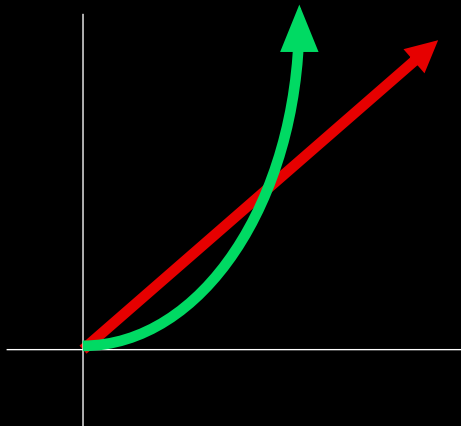
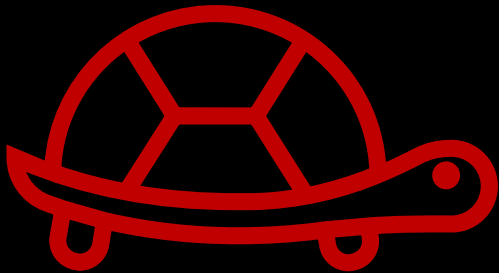
Time,  $y = T(n)$

Inputs:  $n$

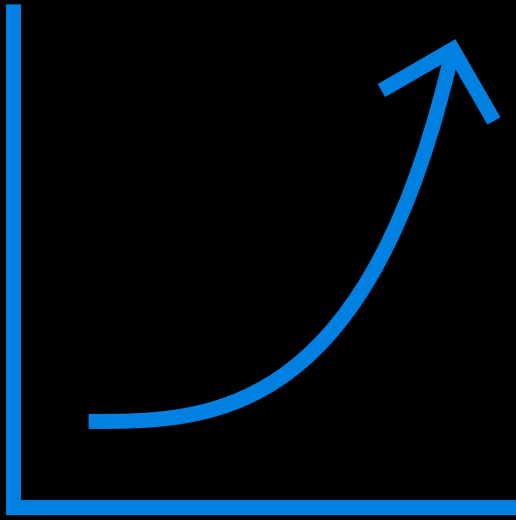
|           | 1 | $\log n$ | $n$       | $n \log n$ | $n^2$       | $n^3$      | $2^n$       | $n!$        |
|-----------|---|----------|-----------|------------|-------------|------------|-------------|-------------|
| 1         | 1 | 0        | 1         | 0          | 1           | 1          | 2           | 1           |
| 10        | 1 | 3        | 10        | 30         | 100         | 1000       | 1024        | 3628800     |
| 100       | 1 | 7        | 100       | 700        | 10000       | 1000000    | 1.26765E+30 | 9.3326E+157 |
| 1000      | 1 | 10       | 1000      | 10000      | 1000000     | 1000000000 | 1.0715E+301 | #NUM!       |
| 10000     | 1 | 13       | 10000     | 130000     | 100000000   | 1E+12      | #NUM!       | #NUM!       |
| 100000    | 1 | 17       | 100000    | 1700000    | 10000000000 | 1E+15      | #NUM!       | #NUM!       |
| 1000000   | 1 | 20       | 1000000   | 20000000   | 1E+12       | 1E+18      | #NUM!       | #NUM!       |
| 10000000  | 1 | 23       | 10000000  | 230000000  | 1E+14       | 1E+21      | #NUM!       | #NUM!       |
| 100000000 | 1 | 27       | 100000000 | 2700000000 | 1E+16       | 1E+24      | #NUM!       | #NUM!       |

Order of Growth gets Faster or Functions rise faster

# Eliminate functions that grow slower

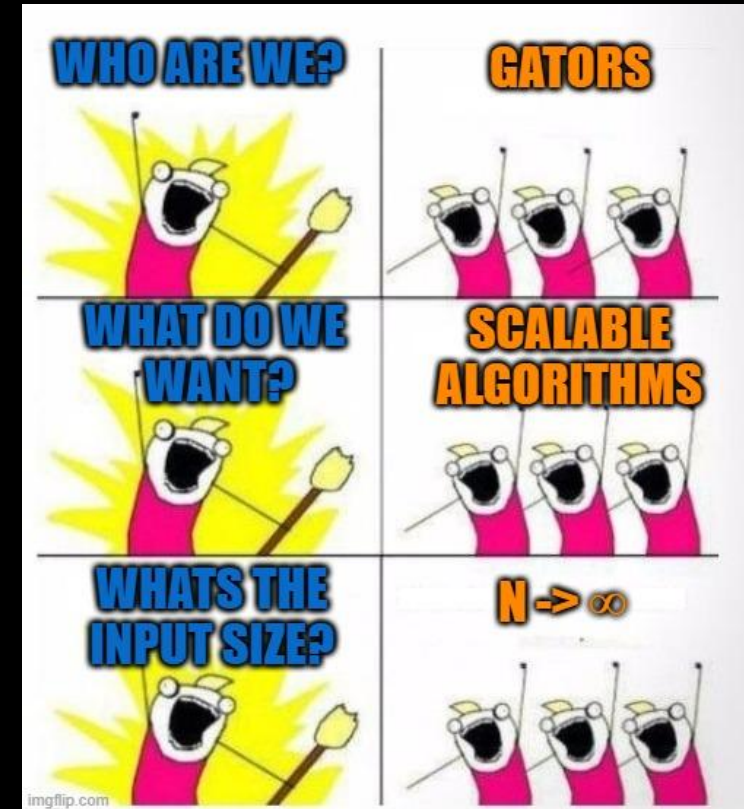
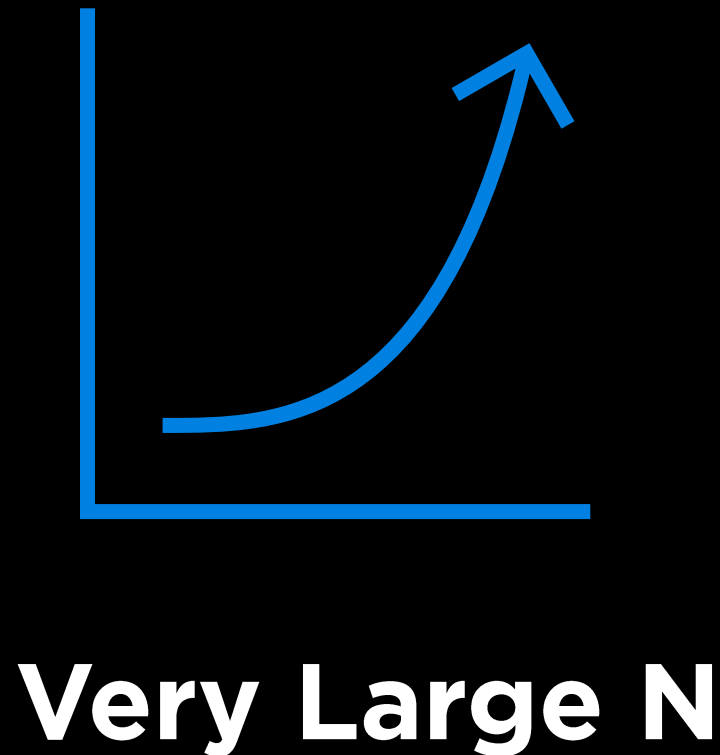


# Approach 3 (Asymptotic Behavior: Order of Growth)



**Very Large  $N$**

# Approach 3 (Asymptotic Behavior: Order of Growth)



<https://imgflip.com/i/3z0jdf>

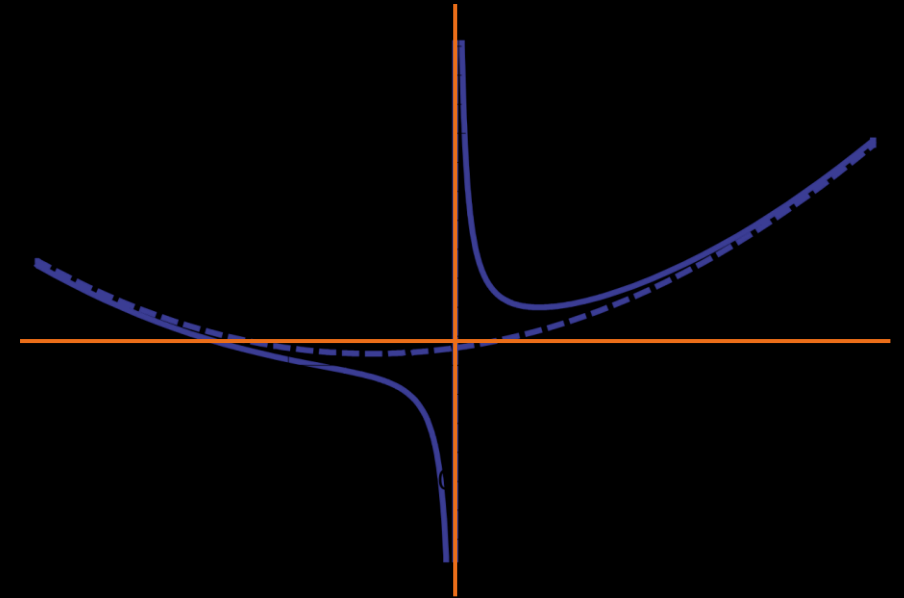
# Notations for Algorithm Complexity

Time - Number of Operations:

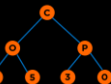
- **Big-O** : Upper Bound
- **Big- $\Omega$**  : Lower Bound
- **Big- $\Theta$**  : Upper + Lower Bound

# Asymptotic Bounding

- Line that approaches a curve but never meets
- Analysis of tail behavior
- $N \rightarrow \text{Infinity}$



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)



# Big O (Visualize)

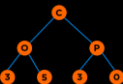
$$T(n) \in O(f(n))$$

- If there exists two positive constants,  $n_0$  and  $c$ , such that  $T(n) \leq c \cdot f(n)$  for all  $n \geq n_0$
- $f(n)$  is an upper bound on performance
- $T(n)$  will grow no faster than constant times  $f(n)$
- Use tighter upper bound

# Big $\Omega$

$$T(n) \in \Omega(g(n))$$

- If there exists two positive constants,  $n_0$  and  $c$ , such that  $T(n) \geq c.g(n)$  for all  $n \geq n_0$
- $g(n)$  is a lower bound on growth rate of  $T(n)$
- $T(n)$  will grow no slower than constant times  $g(n)$
- Use tighter lower bound



# Big $\Theta$

$$T(n) \in \Theta(g(n))$$

- If  $T(n) = O(g(n))$  and  $T(n) = \Omega(g(n))$
- $c_1 \cdot g(n) \leq T(n) \leq c_2 \cdot g(n)$  for all  $n \geq n_0$
- $g(n)$  is a tight upper and lower bound on the growth rate of  $T(n)$

# Big $\Theta$ vs Big $O$ vs Big $\Omega$

|                                | Informal meaning:                                 | Family        | Family Members, $T(n)$            |
|--------------------------------|---------------------------------------------------|---------------|-----------------------------------|
| Big Theta<br>$\Theta(f(N))$    | Order of growth is $f(N)$ .                       | $\Theta(N^2)$ | $N^2/12$<br>$2N^2$<br>$N^2 + 11N$ |
| Big O<br>$O(f(N))$             | Order of growth is less than or equal to $f(N)$ . | $O(N^2)$      | $N^2/2$<br>$N^2 + 1$<br>$\lg(N)$  |
| Big $\Omega$<br>$\Omega(f(N))$ |                                                   |               |                                   |

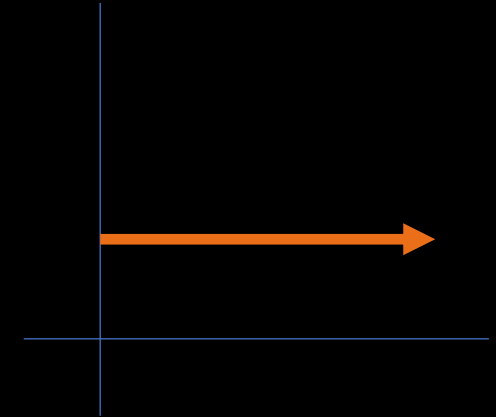
Source: <https://sp19.datastructur.es/index.html>

# Constant Growth Rate, $O(1)$

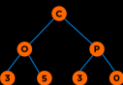
If processing time is independent of the number of inputs  $n$ , the algorithm grows at a constant rate

```
01 int sum(int n)
02 {
03     int sum = 0;
04     sum += n;
05     return sum;
06 }
07
```

| $n$       | $y = f(c)$ |
|-----------|------------|
| 1         | 3          |
| 10        | 3          |
| 100       | 3          |
| 1000      | 3          |
| 10000     | 3          |
| 100000    | 3          |
| 1000000   | 3          |
| 10000000  | 3          |
| 100000000 | 3          |



$$T(n) = 3, T(n) \in O(1)$$

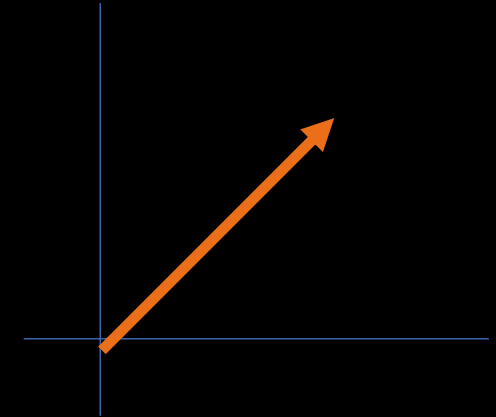


# Linear Growth Rate

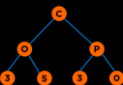
If processing time increases in proportion to the number of inputs  $n$ , the algorithm grows at a linear rate

```
01 int sum(int n)
02 {
03     int sum = 0;
04     for (int i=0; i<n; i++)
05         sum += i+1;
06     return sum;
07 }
```

| $n$       | $y = f(n)$ |
|-----------|------------|
| 1         | 1          |
| 10        | 10         |
| 100       | 100        |
| 1000      | 1000       |
| 10000     | 10000      |
| 100000    | 100000     |
| 1000000   | 1000000    |
| 10000000  | 10000000   |
| 100000000 | 100000000  |



$$T(n) = 3n + 4, T(n) \in O(n), c=4, n_0>4$$



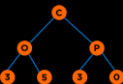
# Quadratic Growth Rate

If processing time increases in proportion to the square of input size  $n$ , the algorithm grows at a quadratic rate

```
01 bool find(int n[][], int t)
02 {
03     int i, j;
04     for(i=0; i < n.size; i++)
05         for(j=0; j < n.size; j++)
06             if (x[i][j] == t)
07                 return true;
08     return false;
09 }
```

$$T(n) = 3n^2 + 4n + 5$$

| Operation         | Count              |
|-------------------|--------------------|
| int i, j;         | 2                  |
| i=0               | 1                  |
| i < n             | n+1                |
| i++               | n                  |
| j=0               | n                  |
| j < n             | $n(n+1) = n^2 + n$ |
| j++               | $n^2$              |
| x[i][j] == t      | $n^2$              |
| return true/false | 1                  |
| T(n)              | $3n^2 + 4n + 5$    |

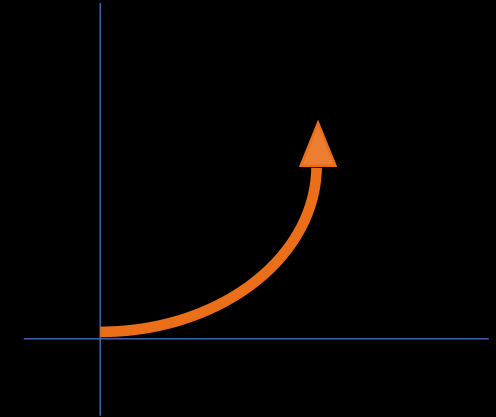


# Quadratic Growth Rate

If processing time increases in proportion to the square of input size  $n$ , the algorithm grows at a quadratic rate

```
01 bool find(int n[][], int t)
02 {
03     int i, j;
04     for(i=0; i < n.size; i++)
05         for(j=0; j < n.size; j++)
06             if (x[i][j] == t)
07                 return true;
08     return false;
09 }
```

| $n$       | $y = f(n^2)$ |
|-----------|--------------|
| 1         | 1            |
| 10        | 100          |
| 100       | 10000        |
| 1000      | 1000000      |
| 10000     | 100000000    |
| 100000    | 10000000000  |
| 1000000   | 1E+12        |
| 10000000  | 1E+14        |
| 100000000 | 1E+16        |



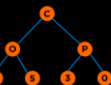
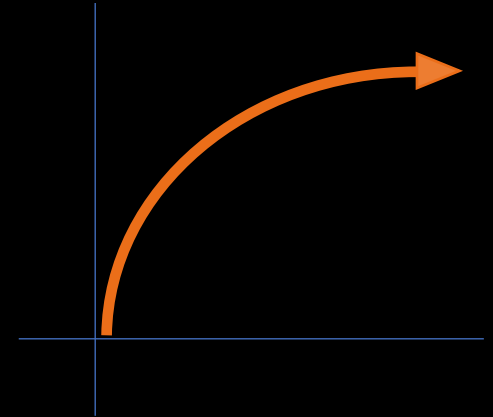
$$T(n) = 3n^2 + 4n + 5, T(n) \in O(n^2), c=?, n_0>?$$

# Logarithmic Growth Rate

If processing time increases in proportion to the  $\log n$ , the algorithm grows at a logarithmic rate

```
01 int sum(int n)
02 {
03     int sum = 0;
04     for (int i=1; i<=n; i*=2)
05         sum += i;
06     return sum;
07 }
```

| $n$       | $y = f(\log_2 n)$ |
|-----------|-------------------|
| 1         | 0                 |
| 10        | 3                 |
| 100       | 7                 |
| 1000      | 10                |
| 10000     | 13                |
| 100000    | 17                |
| 1000000   | 20                |
| 10000000  | 23                |
| 100000000 | 27                |

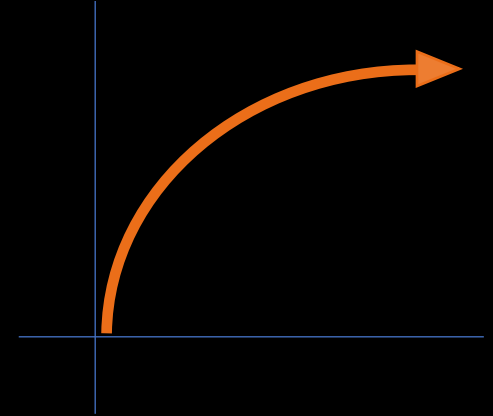


# Logarithmic Growth Rate

If processing time increases in proportion to the  $\log n$ , the algorithm grows at a logarithmic rate

```
01 int sum(int n)
02 {
03     int sum = 0;
04     for (int i=1; i<=n; i*=2)
05         sum += i;
06     return sum;
07 }
```

| n         | y = f(log <sub>2</sub> n) |
|-----------|---------------------------|
| 1         | 0                         |
| 10        | 3                         |
| 100       | 7                         |
| 1000      | 10                        |
| 10000     | 13                        |
| 100000    | 17                        |
| 1000000   | 20                        |
| 10000000  | 23                        |
| 100000000 | 27                        |



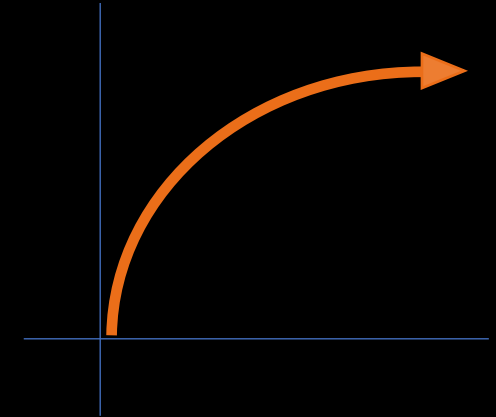
|                              |                |                |                |                |                |   |   |   |   |
|------------------------------|----------------|----------------|----------------|----------------|----------------|---|---|---|---|
| n                            | 1              | 2              | 4              | 8              | 16             | . | . | . | ? |
| n (in powers of two)         | 2 <sup>0</sup> | 2 <sup>1</sup> | 2 <sup>2</sup> | 2 <sup>3</sup> | 2 <sup>4</sup> | . | . | . | ? |
| # of times the loop executes | 1              | 2              | 3              | 4              | 5              | . | . | . | k |

# Logarithmic Growth Rate

If processing time increases in proportion to the  $\log n$ , the algorithm grows at a logarithmic rate

```
01 int sum(int n)
02 {
03     int sum = 0;
04     for (int i=1; i<=n; i*=2)
05         sum += i;
06     return sum;
07 }
```

| n         | y = f(log <sub>2</sub> n) |
|-----------|---------------------------|
| 1         | 0                         |
| 10        | 3                         |
| 100       | 7                         |
| 1000      | 10                        |
| 10000     | 13                        |
| 100000    | 17                        |
| 1000000   | 20                        |
| 10000000  | 23                        |
| 100000000 | 27                        |



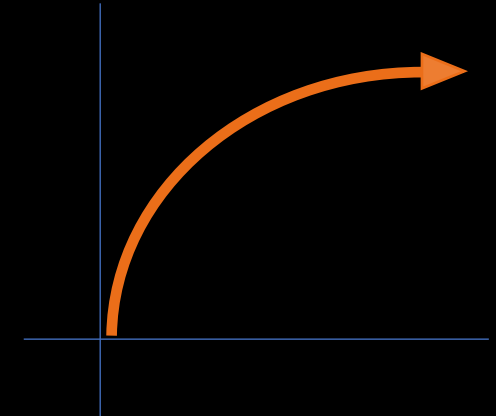
|                              |       |       |       |       |       |   |   |   |           |
|------------------------------|-------|-------|-------|-------|-------|---|---|---|-----------|
| n                            | 1     | 2     | 4     | 8     | 16    | . | . | . | $2^{k-1}$ |
| n (in powers of two)         | $2^0$ | $2^1$ | $2^2$ | $2^3$ | $2^4$ | . | . | . | $2^{k-1}$ |
| # of times the loop executes | 1     | 2     | 3     | 4     | 5     | . | . | . | k         |

# Logarithmic Growth Rate

If processing time increases in proportion to the  $\log n$ , the algorithm grows at a logarithmic rate

```
01 int sum(int n)
02 {
03     int sum = 0;
04     for (int i=1; i<=n; i*=2)
05         sum += i;
06     return sum;
07 }
```

| n         | y = f(log <sub>2</sub> n) |
|-----------|---------------------------|
| 1         | 0                         |
| 10        | 3                         |
| 100       | 7                         |
| 1000      | 10                        |
| 10000     | 13                        |
| 100000    | 17                        |
| 1000000   | 20                        |
| 10000000  | 23                        |
| 100000000 | 27                        |



|                              |       |       |       |       |       |   |   |   |           |
|------------------------------|-------|-------|-------|-------|-------|---|---|---|-----------|
| n                            | 1     | 2     | 4     | 8     | 16    | . | . | . | $2^{k-1}$ |
| n (in powers of two)         | $2^0$ | $2^1$ | $2^2$ | $2^3$ | $2^4$ | . | . | . | $2^{k-1}$ |
| # of times the loop executes | 1     | 2     | 3     | 4     | 5     | . | . | . | k         |

We know that  $i \leq n$ ; Therefore, at iteration  $k$ ,  $i = 2^{k-1}$ ; Since the loop continues while  $i \leq n$ , we have:  $2^{k-1} \leq n$ ;  $k \leq (\log_2 n + 1)$ ;  $k \propto \log_2 n$ ;

# Different Growth Rates

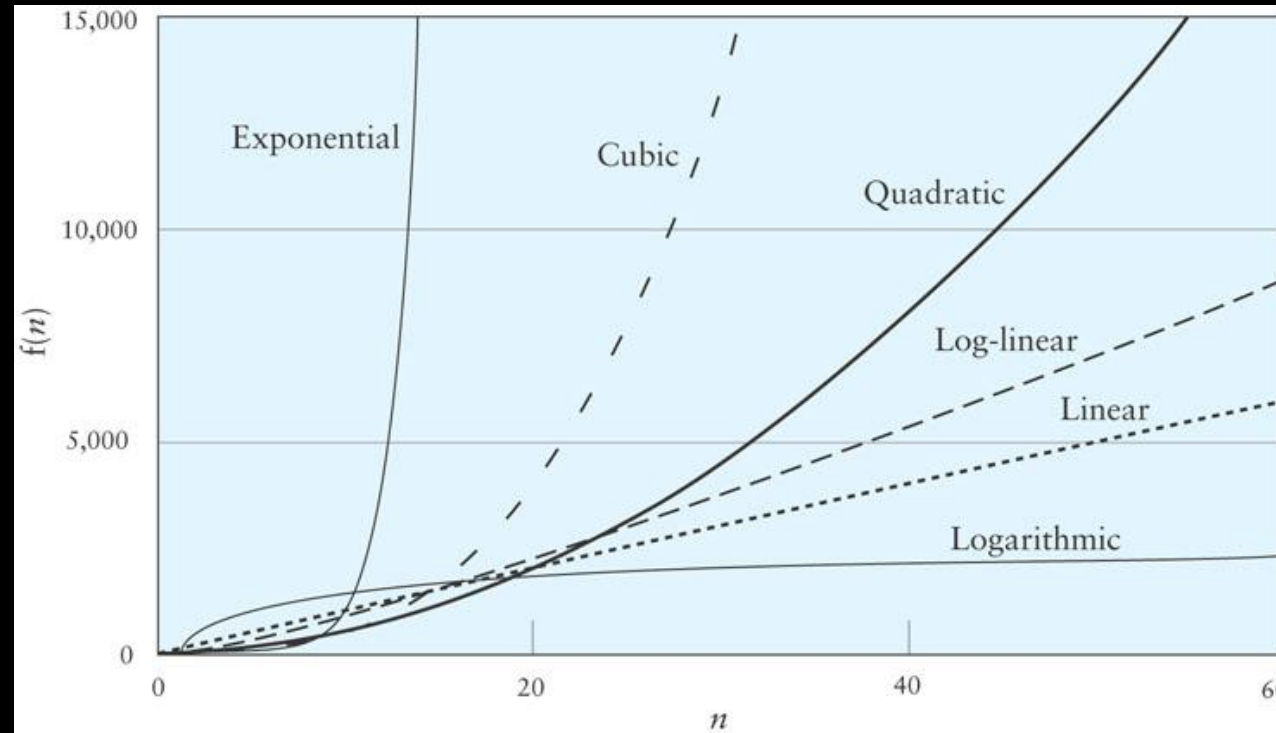
Time

| Inputs: n | $O(1)$    | $O(\log n)$ | $O(n)$    | $O(n \log n)$ | $O(n^2)$    | $O(n^3)$   | $O(2^n)$    | $O(n!)$     |
|-----------|-----------|-------------|-----------|---------------|-------------|------------|-------------|-------------|
|           | 1         | 0           | 1         | 0             | 1           | 1          | 2           | 1           |
|           | 10        | 3           | 10        | 10            | 100         | 1000       | 1024        | 3628800     |
|           | 100       | 7           | 100       | 200           | 10000       | 1000000    | 1.26765E+30 | 9.3326E+157 |
|           | 1000      | 10          | 1000      | 3000          | 1000000     | 1000000000 | 1.0715E+301 | #NUM!       |
|           | 10000     | 13          | 10000     | 40000         | 100000000   | 1E+12      | #NUM!       | #NUM!       |
|           | 100000    | 17          | 100000    | 500000        | 10000000000 | 1E+15      | #NUM!       | #NUM!       |
|           | 1000000   | 20          | 1000000   | 6000000       | 1E+12       | 1E+18      | #NUM!       | #NUM!       |
|           | 10000000  | 23          | 10000000  | 70000000      | 1E+14       | 1E+21      | #NUM!       | #NUM!       |
|           | 100000000 | 27          | 100000000 | 800000000     | 1E+16       | 1E+24      | #NUM!       | #NUM!       |

Order of Growth gets Faster, Complexity increases

Constant < Logarithmic < Linear < Loglinear < Polynomial < Exponential < Factorial

# Different Growth Rates



<http://bigocheatsheet.com/>

# Tips for Asymptotic Analysis

## (Big O)

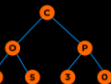
# Tip #1: Addition (Independence)

```
1. void func1(int n, int m)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 0; j < m; j++)
7.         cout << j;
8. }
```

# Tip #1: Addition (Independence)

```
1. void func1(int n, int m)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 0; j < m; j++)
7.         cout << j;
8. }
```

$$T(n, m) \in O(n+m)$$



# Tip #2: Drop Constant Multipliers

```
1. void func2(int n)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 0; j < n; j++)
7.         cout << j;
8. }
```

# Tip #2: Drop Constant Multipliers

```
1. void func2(int n)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 0; j < n; j++)
7.         cout << j;
8. }
```

$T(n) \in O(n+n)$   
 $\in O(2n)$   
 $\sim O(n)$

# Tip #3: Different Input Variables

```
1. void func3(int x, int y)
2. {
3.     for (int i = 0; i < x; i++)
4.         cout << i;
5.
6.     for (int j = 0; j < y; j++)
7.         cout << j;
8. }
```

# Tip #3: Different Input Variables

```
1. void func3(int x, int y)
2. {
3.     for (int i = 0; i < x; i++)
4.         cout << i;
5.
6.     for (int j = 0; j < y; j++)
7.         cout << j;
8. }
```

$$T(x, y) \in O(x+y)$$

**Describe what the variable is, Always!**

**Example:  $O(x)$  where  $x$  is the size or length of a string**

# Tip #4a: Drop Lower Order Terms with Similar Growth Rates

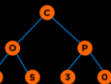
```
1. void func4a(int n)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 1; j < n; j*=2)
7.         cout << j;
8. }
```

# Tip #4a: Drop Lower Order Terms with Similar Growth Rates

```
1. void func4a(int n)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 1; j < n; j*=2)
7.         cout << j;
8. }
```

$$T(n) \in O(n + \log_2 n) \\ \sim O(n)^*$$

\*Both variables are  $n$  and grow at the same rate.



# Tip #4b: Drop Lower Order Terms with Similar Growth Rates

```
1. void func4b(int n, int m)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 1; j < m; j*=2)
7.         cout << j;
8. }
```

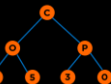
# Tip #4b: Drop Lower Order Terms with Similar Growth Rates

```
1. void func4b(int n, int m)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 1; j < m; j*=2)
7.         cout << j;
8. }
```

$$T(n, m) \in O(n + \log_2 m) \\ \sim O(n)^*$$

\*Assuming  $n$  and  $m$  are growing at the same rate.

If you are **given** in the question that  $n$  and  $m$  are growing at the same rate **or** if you **assume** they are growing at the same rate, then simplifying is fine

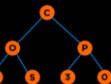


# Tip #4c: Do not drop Lower Order Terms with different Growth Rates

```
1. void func4c(int n, int m)
2. {
3.     for (int i = 0; i < n; i++)
4.         cout << i;
5.
6.     for (int j = 1; j < m; j*=2)
7.         cout << j;
8. }
```

$$T(n, m) \in O(n + \log_2 m)^*$$

\*Assuming no relationship is given between  $n$  and  $m$ .



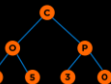
# Mentimeter

## Menti.com

# Mentimeter

```
for(int i = 1; i < n; i++)  
    for(int j=n ; j > 0; j=j/2)  
        print "Cop3530"
```

$O(n \log n)$



# Logarithmic growth

```
for(i = 1; i <= n; i *= 2)
```

```
for(i = n; i >= 1; i /= 2)
```

[https://en.wikipedia.org/wiki/1\\_2\\_3\\_4\\_%E2%8B%AF](https://en.wikipedia.org/wiki/1_2_3_4_%E2%8B%AF)

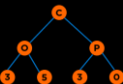
[https://en.wikipedia.org/wiki/1\\_2\\_4\\_8\\_%E2%8B%AF](https://en.wikipedia.org/wiki/1_2_4_8_%E2%8B%AF)

[https://en.wikipedia.org/wiki/1/2\\_1/4\\_1/8\\_1/16\\_%E2%8B%AF](https://en.wikipedia.org/wiki/1/2_1/4_1/8_1/16_%E2%8B%AF)

# Mentimeter

```
for (int i = 100; i > -1; i--)  
    for (int j = i; j > 1; j/=2)  
        print("viola")
```

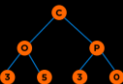
$O(1)$



# Mentimeter

```
for(int i = n; i > 0; i /= 2)
    for(int j = 1; j < i; j++)
        sum += 1;
```

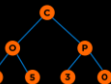
$O(n)$



# Mentimeter

```
// This is A  
for(int i=1; i<n; i*=2);  
  
// This is B  
for(int i=1; i<n; i*=3);
```

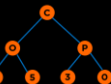
They both take same time in terms of Big O



# Mentimeter

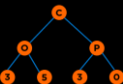
```
// This is A  
for(int i=1; i<n; i*=2);  
  
// This is B  
for(int i=1; i<n; i*=3);
```

B will be faster in terms of execution  
time/simulation



# Enter Data

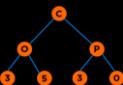
| <b>Best Case</b>   | <b>Average Case</b>                        | <b>Worst Case</b>   |
|--------------------|--------------------------------------------|---------------------|
| <b>Lowest cost</b> | <b>Average cost for all <math>n</math></b> | <b>Highest cost</b> |



# Enter Data

| Best Case   | Average Case             | Worst Case   |
|-------------|--------------------------|--------------|
| Lowest cost | Average cost for all $n$ | Highest cost |

- **Average/Best/Worst case** measure actual costs at a specific input instance.
- You can define a specific order of instance but **cannot propose variability in input size**.
  - In general, calculating best case time complexity under the assumption that the data structure has a small size, example: when an array has size 1 or the tree is empty **should be avoided**. The complexity is calculated without thinking about the size of input. But it is perfectly fine to think about the properties of input or data structure such as data is sorted, height will always be proportional to  $\log n$  for a balanced tree, etc.
  - **Asymptotic analysis** assumes  **$n$  is very large**. Whether it be big O, theta, or omega, it always refers to the case of very large  $n$ . The best / average / worst cases arise in different structural cases, exclusive of size.
- **Growth Rate** measures change in costs.



# Recommended Readings

- <https://dev.to/sherryummen/asymptotic-notations-b-o-o-t-big-o-big-omega-big-theta-49e7>
- [Chapter 8.10 OpenDSA: Common Misunderstanding](#)
- <https://cs.stackexchange.com/questions/23068/how-do-o-and-%CE%A9-relate-to-worst-and-best-case>
- <https://qr.ae/pNyFxo>
- <https://cs.stackexchange.com/questions/23593/is-there-a-system-behind-the-magic-of-algorithm-analysis>
- <https://stackoverflow.com/questions/25593619/why-small-theta-asymtotic-notation-doesnt-exists/54542603>

# Useful series

$$1 + 2 + 3 + 4 + \dots + n = n \cdot (n+1) / 2$$

$$1 + 2 + 4 + 8 + \dots + 2^k = 2^{k+1} - 1$$

$$n + n/2 + n/4 + n/8 + \dots + 1 = 2n - 1$$

[https://en.wikipedia.org/wiki/1 %2B 2 %2B 3 %2B 4 %2B %E2%8B%AF](https://en.wikipedia.org/wiki/1_%2B_2_%2B_3_%2B_4_%2B_%E2%8B%AF)

[https://en.wikipedia.org/wiki/1 %2B 2 %2B 4 %2B 8 %2B %E2%8B%AF](https://en.wikipedia.org/wiki/1_%2B_2_%2B_4_%2B_8_%2B_%E2%8B%AF)

[https://en.wikipedia.org/wiki/1/2 %2B 1/4 %2B 1/8 %2B 1/16 %2B %E2%8B%AF](https://en.wikipedia.org/wiki/1/2_%2B_1/4_%2B_1/8_%2B_1/16_%2B_%E2%8B%AF)

# Linear Search

```
1.  #include <iostream>
2.  #include <string>
3.
4.  std::string linearSearch(const std::string arr[], int size, std::string t)
5.  {
6.      for(int i = 0; i < size; i++)
7.      {
8.          if(arr[i].compare(t) == 0)
9.              return std::string("found at index = ") + std::to_string(i);
10.     }
11.     return std::string("not found");
12. }
13.
14. int main()
15. {
16.     std::string arr[] = {"hello", "world", "cop3530", "cop3502"};
17.     std::string target = "curious";
18.     int size = sizeof(arr) / sizeof(arr[0]);
19.     std::cout << (linearSearch(arr, size, target));
20.     return 0;
21. }
```

# Linear Search: Assuming a large array

```
1.  #include <iostream>
2.  #include <string>
3.
4.  std::string linearSearch(const std::string arr[], int size, std::string t)
5.  {
6.      for(int i = 0; i < size; i++)
7.      {
8.          if(arr[i].compare(t) == 0)
9.              return std::string("found at index = ") + std::to_string(i);
10.     }
11.     return std::string("not found");
12. }
13.
14. int main()
15. {
16.     std::string arr[] = {"hello", "world", "cop3530", "cop3502", ...};
17.     std::string target = "curious";
18.     int size = sizeof(arr) / sizeof(arr[0]);
19.     std::cout << (linearSearch(arr, size, target));
20.     return 0;
21. }
```

# Linear Search: Assuming a large array

```
1.  #include <iostream>
2.  #include <string>
3.
4.  std::string linearSearch(const std::string arr[], int size, std::string t)
5.  {
6.      for(int i = 0; i < size; i++)
7.      {
8.          if(arr[i].compare(t) == 0)
9.              return std::string("found at index = ") + std::to_string(i);
10.     }
11.     return std::string("not found");
12. }
13.
14. int main()
15. {
16.     std::string arr[] = {"hello", "world", "cop3530", "cop3502", ...};
17.     std::string target = "curious";
18.     int size = sizeof(arr) / sizeof(arr[0]);
19.     std::cout << (linearSearch(arr, size, target));
20.     return 0;
21. }
```

Size of array = **size**  
Max length of any string = **s**

Length of target = **t**

constant

...

# Linear Search: Assuming a large array

```
1.  #include <iostream>
2.  #include <string>
3.
4.  std::string linearSearch(const std::string arr[], int size, std::string t)
5.  {
6.      for(int i = 0; i < size; i++)
7.      {
8.          if(arr[i].compare(t) == 0)
9.              return std::string("found at index = ") + std::to_string(i);
10.     }
11.     return std::string("not found");
12. }
13.
14. int main()
15. {
16.     std::string arr[] = {"hello", "world", "cop3530", "cop3502", ...};
17.     std::string target = "curious";
18.     int size = sizeof(arr) / sizeof(arr[0]);
19.     std::cout << (linearSearch(arr, size, target));
20.     return 0;
21. }
```

linear and dependent on **size**

linear and dependent on **min(t, s)**

constant

constant

Size of array = **size**

Max length of any string = **s**

Length of target = **t**

constant

...

# Linear Search: Assuming a large array

```
1.  #include <iostream>
2.  #include <string>
3.
4.  std::string linearSearch(const std::string arr[], int size, std::string t)
5.  {
6.      for(int i = 0; i < size; i++)
7.      {
8.          if(arr[i].compare(t) == 0)
9.              return std::string("found at index = ") + std::to_string(i);
10.     }
11.     return std::string("not found");
12. }
13.
14. int main()
15. {
16.     std::string arr[] = {"hello", "world", "cop3530", "cop3502", ...};
17.     std::string target = "curious";
18.     int size = sizeof(arr) / sizeof(arr[0]);
19.     std::cout << (linearSearch(arr, size, target));
20.     return 0;
21. }
```

$(\text{size} * \min(t, s)) + c$

linear and dependent on **size**

linear and dependent on **min(t, s)**

constant

constant

Size of array = **size**

Max length of any string = **s**

Length of target = **t**

constant

$(\text{size} * \min(t, s)) + c$

$O(\text{size} * t)$ , where **size** is size of array, **t** is size of target string, **s** is the max length of all strings in the array, and we are assuming that **t** and **s** grow at same rates.

<https://onlinegdb.com/oPeGj9PmNm>

# Linear Search: Assuming a large array

```
1.  #include <iostream>
2.  #include <string>
3.
4.  std::string linearSearch(const std::string arr[], int size, std::string t)
5.  {
6.      for(int i = 0; i < size; i++)
7.      {
8.          if(arr[i].compare(t) == 0)
9.              return std::string("found at index = ") + std::to_string(i);
10.     }
11.     return std::string("not found");
12. }
13.
14. int main()
15. {
16.     std::string arr[] = {"hello", "world", "cop3530", "cop3502", ...};
17.     std::string target = "curious";
18.     int size = sizeof(arr) / sizeof(arr[0]);
19.     std::cout << (linearSearch(arr, size, target));
20.     return 0;
21. }
```

$O(\text{size} \cdot t)$ , where  $\text{size}$  is size of array,  $t$  is size of target string,  $s$  is the max length of all strings in the array, and we are assuming that  $t$  and  $s$  grow at same rates.

## Clarification:

This program will yield an equation,

$$T(\text{size}, t) = (\text{size} * \min(t, s)) + X_c$$

This function  $T(\text{size}, t)$  will be an element of  $O(\text{size} \cdot t)$ ,  $O(\text{size} \cdot t \cdot t)$ ,  $O(\text{size} \cdot \text{size} \cdot t)$ , ... and several other functions.

When we talk about Big O, we want a measure to describe the upper bound. For the sake of the course, we seek the tightest upper bound.

# Linear Search: Assuming a large array

```
1.  #include <iostream>
2.  #include <string>
3.
4.  std::string linearSearch(const std::string arr[], int size, std::string t)
5.  {
6.      for(int i = 0; i < size; i++)
7.      {
8.          if(arr[i].compare(t) == 0)
9.              return std::string("found at index = ") + std::to_string(i);
10.     }
11.     return std::string("not found");
12. }
13.
14. int main()
15. {
16.     std::string arr[] = {"hello", "world", "cop3530", "cop3502", ...};
17.     std::string target = "curious";
18.     int size = sizeof(arr) / sizeof(arr[0]);
19.     std::cout << (linearSearch(arr, size, target));
20.     return 0;
21. }
```

$O(\text{size} \cdot t)$ , where  $\text{size}$  is size of array,  $t$  is size of target string,  $s$  is the max length of all strings in the array, and we are assuming that  $t$  and  $s$  grow at same rates.

## Clarification:

This program will yield an equation,

$$T(\text{size}, t) = (\text{size} * \min(t, s)) + X_c$$

This function  $T(\text{size}, t)$  will be an element of  $\Omega(\text{size} \cdot t)$ ,  $\Omega(\text{size})$ ,  $\Omega(t)$ ,  $\Omega(1)$  ... and several other functions.

When we talk about Big  $\Omega$ , we want a measure to describe the lower bound. For the sake of the course, we seek the tightest lower bound.

# Linear Search: Assuming a large array

```
1.  #include <iostream>
2.  #include <string>
3.
4.  std::string linearSearch(const std::string arr[], int size, std::string t)
5.  {
6.      for(int i = 0; i < size; i++)
7.      {
8.          if(arr[i].compare(t) == 0)
9.              return std::string("found at index = ") + std::to_string(i);
10.     }
11.     return std::string("not found");
12. }
13.
14. int main()
15. {
16.     std::string arr[] = {"hello", "world", "cop3530", "cop3502", ...};
17.     std::string target = "curious";
18.     int size = sizeof(arr) / sizeof(arr[0]);
19.     std::cout << (linearSearch(arr, size, target));
20.     return 0;
21. }
```

$O(\text{size} * t)$ , where size is size of array, t is size of target string, s is the max length of all strings in the array, and we are assuming that t and s grow at same rates.

## Clarification:

This program will yield an equation,

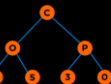
$$T(\text{size}, t) = (\text{size} * \min(t, s)) + X_c$$

In this program, the time complexity of the code is

- $O(\text{size} * t)$
- $\Omega(\text{size} * t)$
- $\Theta(\text{size} * t)$

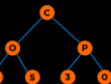
# Peak Finding

- **Input:**
  - You are given an array of numbers
  - The data is randomly sorted
- **Output:**
  - A Peak value
  - Peak is a number such that it is **greater than or equal to both** of its adjacent elements
  - In case of the boundary values, a peak must be greater than or equal to the one adjacent element.



# Peak Finding: Case 1

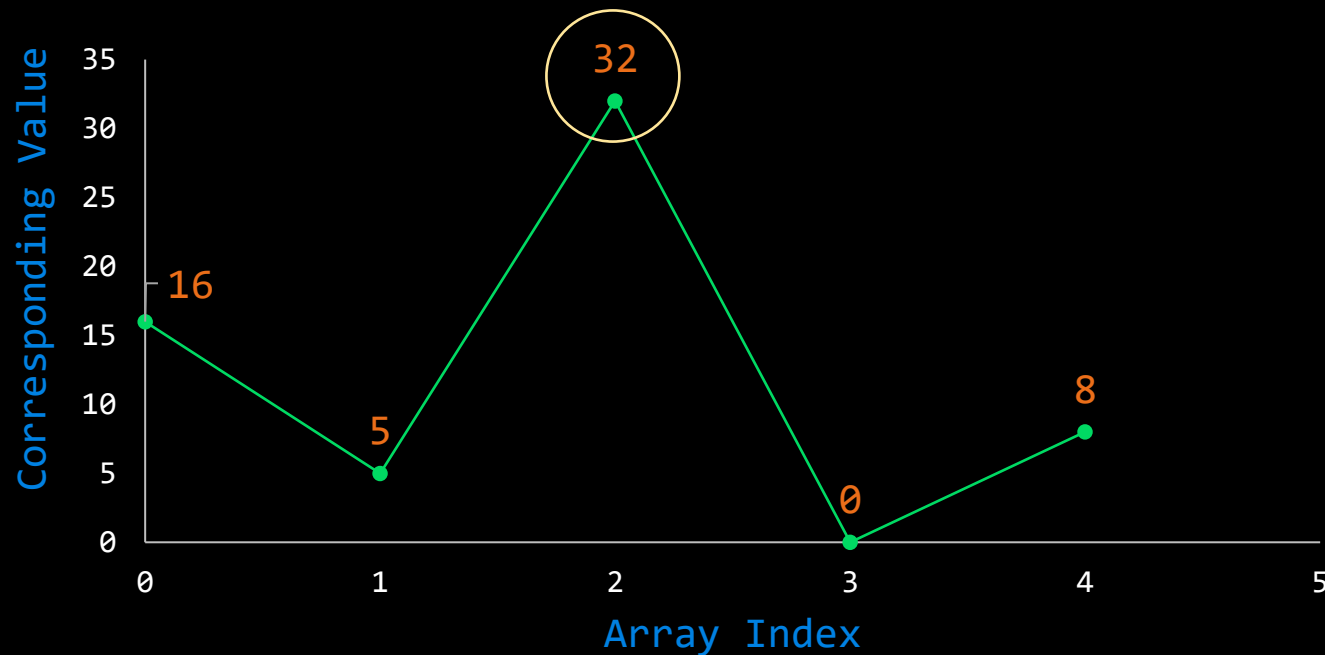
|    |   |    |   |   |
|----|---|----|---|---|
| 16 | 5 | 32 | 0 | 8 |
|----|---|----|---|---|



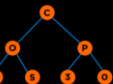
# Peak Finding: Case 1

|    |   |    |   |   |
|----|---|----|---|---|
| 16 | 5 | 32 | 0 | 8 |
|----|---|----|---|---|

2D Representation of the Problem



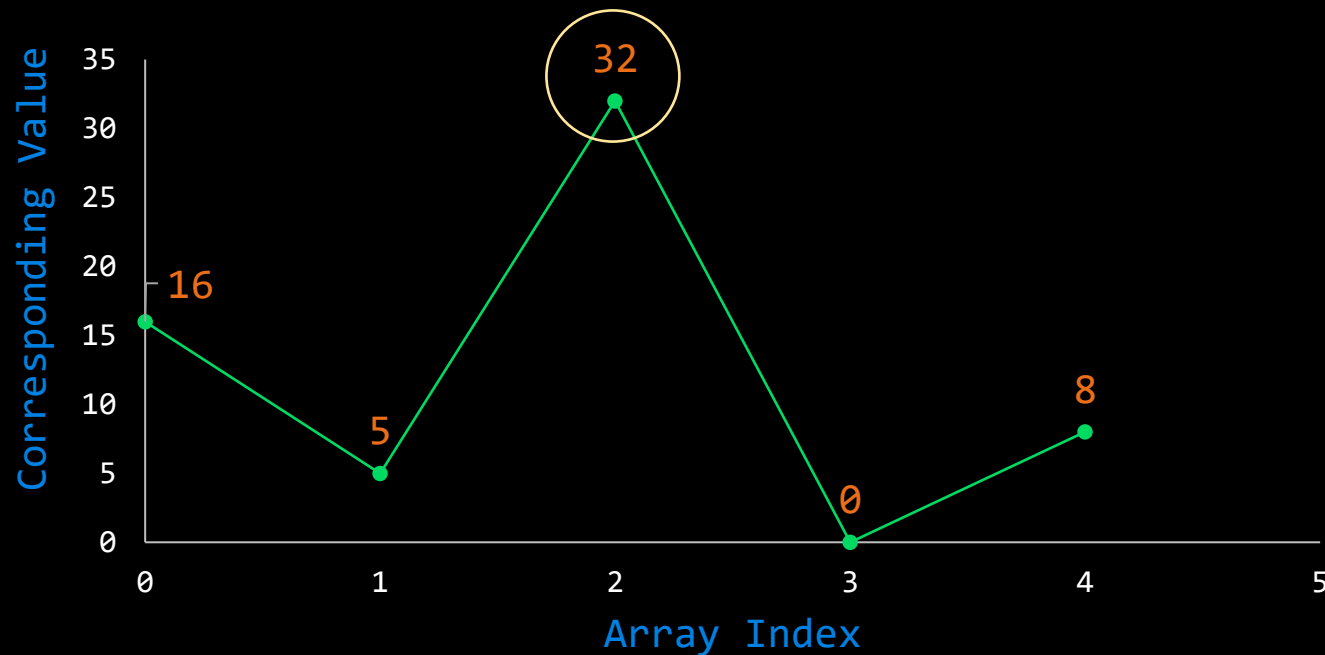
**Case 1: Central Element larger than both adjacent elements:**



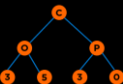
# Peak Finding: Case 1

|    |   |    |   |   |
|----|---|----|---|---|
| 16 | 5 | 32 | 0 | 8 |
|----|---|----|---|---|

2D Representation of the Problem



**Case 1: Central Element larger than both adjacent elements: Peak**

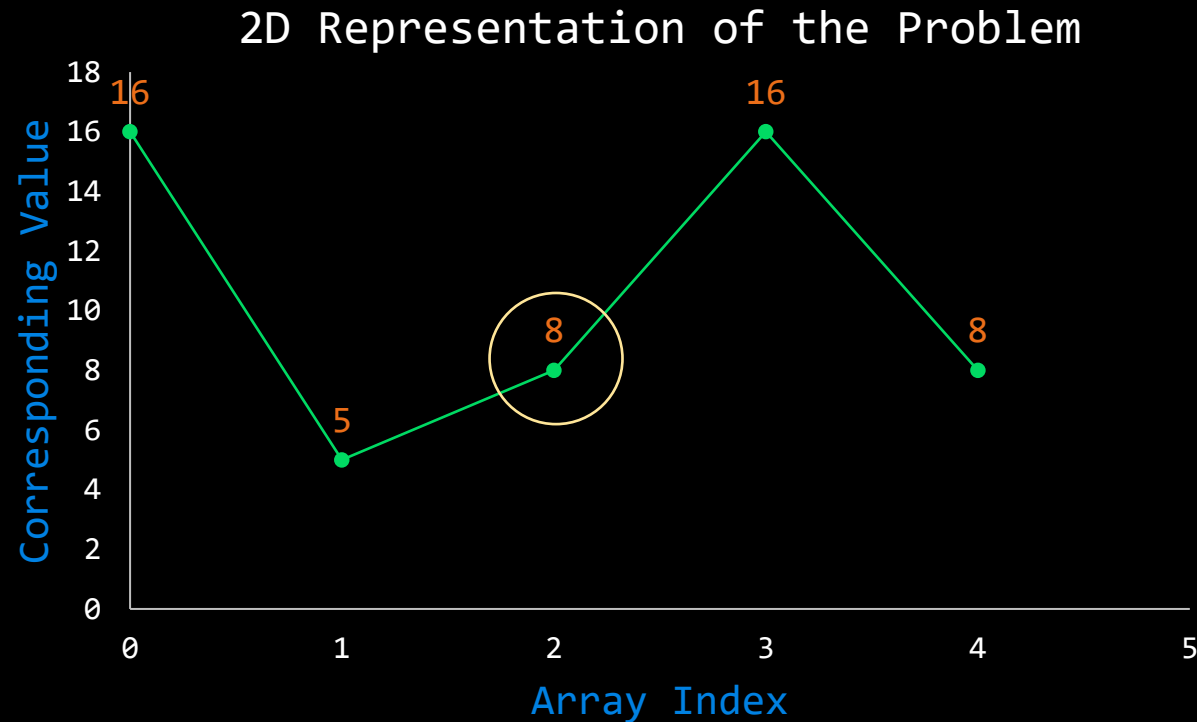


# Peak Finding: Case 2

|    |   |   |    |   |
|----|---|---|----|---|
| 16 | 5 | 8 | 16 | 8 |
|----|---|---|----|---|

# Peak Finding: Case 2

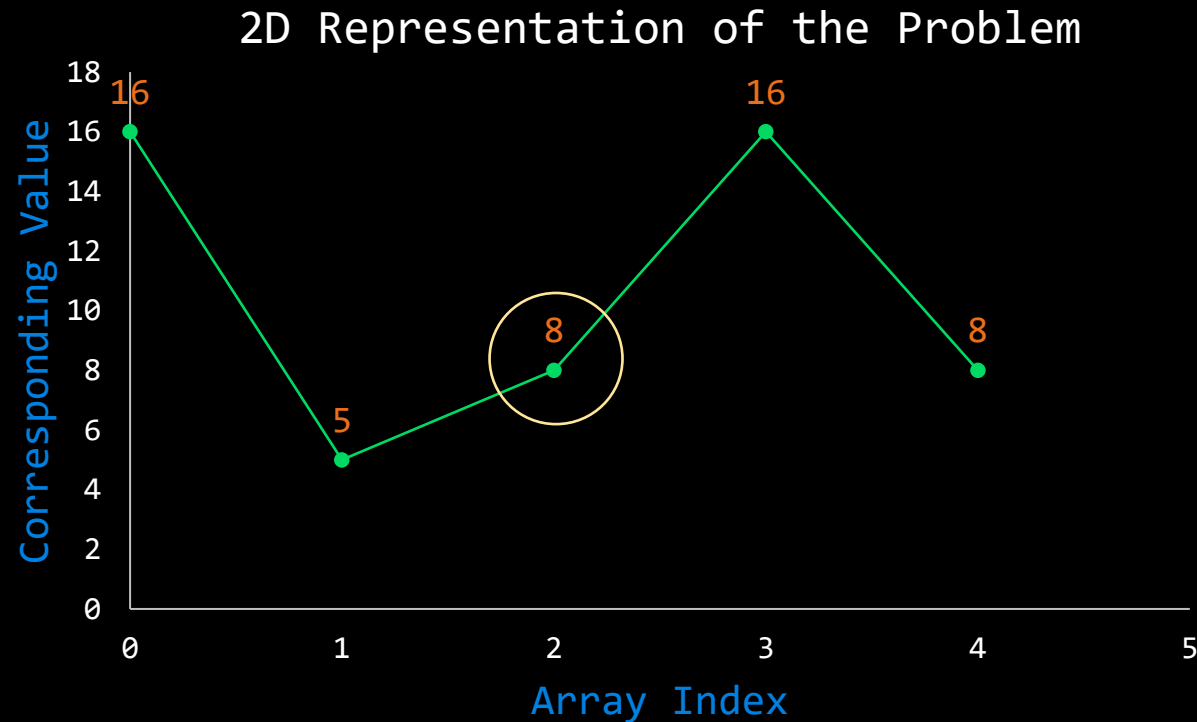
|    |   |   |    |   |
|----|---|---|----|---|
| 16 | 5 | 8 | 16 | 8 |
|----|---|---|----|---|



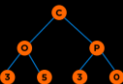
**Case 2: Central Element larger than left element and smaller than right element:**

# Peak Finding: Case 2

|    |   |   |    |   |
|----|---|---|----|---|
| 16 | 5 | 8 | 16 | 8 |
|----|---|---|----|---|

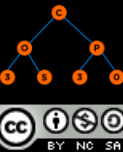


**Case 2: Central Element larger than left element and smaller than right element: Keep going right**



# Peak Finding: Case 3

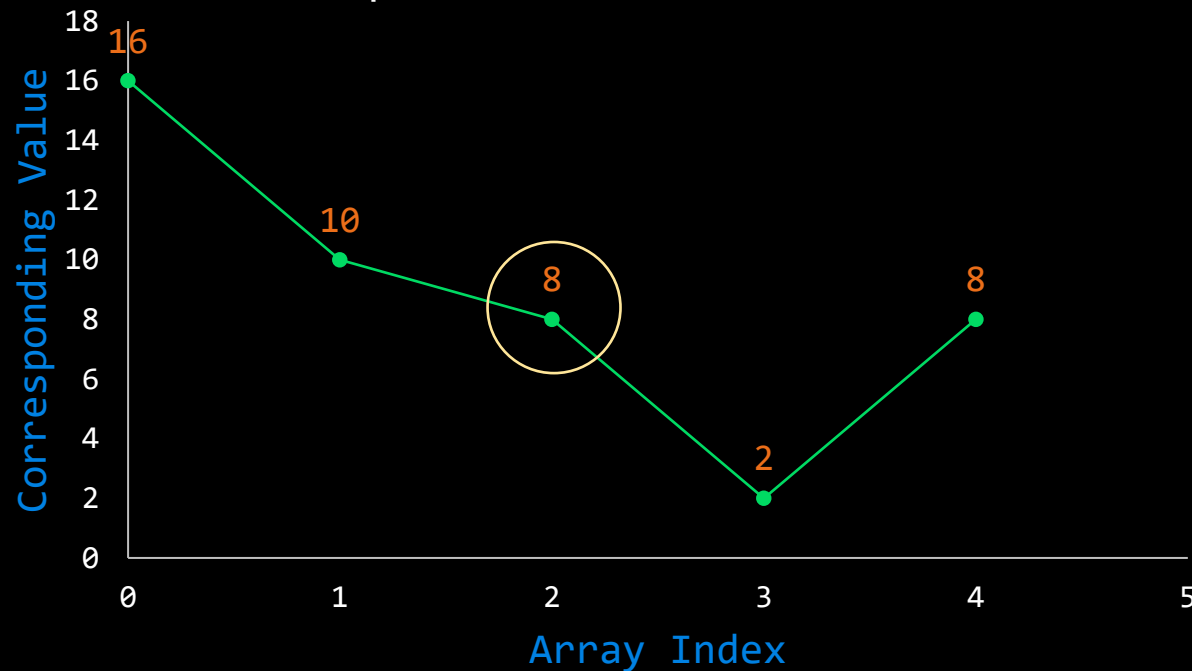
|    |    |   |   |   |
|----|----|---|---|---|
| 16 | 10 | 8 | 2 | 8 |
|----|----|---|---|---|



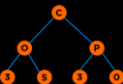
# Peak Finding: Case 3

|    |    |   |   |   |
|----|----|---|---|---|
| 16 | 10 | 8 | 2 | 8 |
|----|----|---|---|---|

2D Representation of the Problem



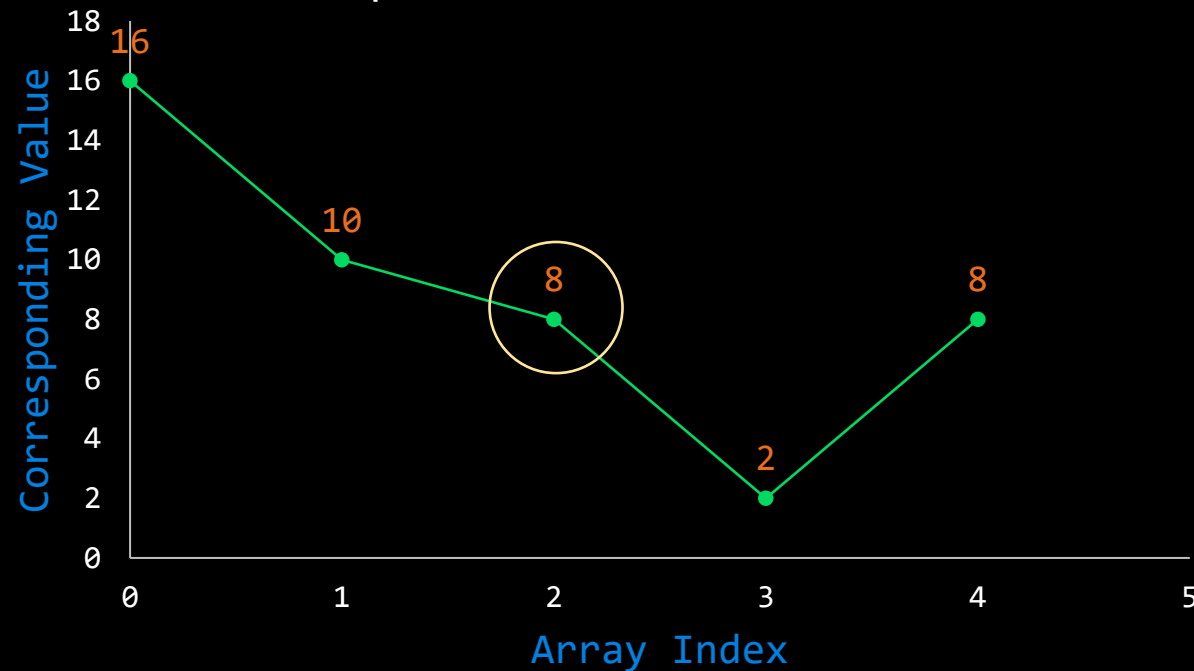
**Case 3:** Central Element larger than right element and smaller than left element:



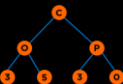
# Peak Finding: Case 3

|    |    |   |   |   |
|----|----|---|---|---|
| 16 | 10 | 8 | 2 | 8 |
|----|----|---|---|---|

2D Representation of the Problem

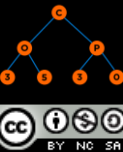


**Case 3: Central Element larger than right element and smaller than left element: Keep going left**



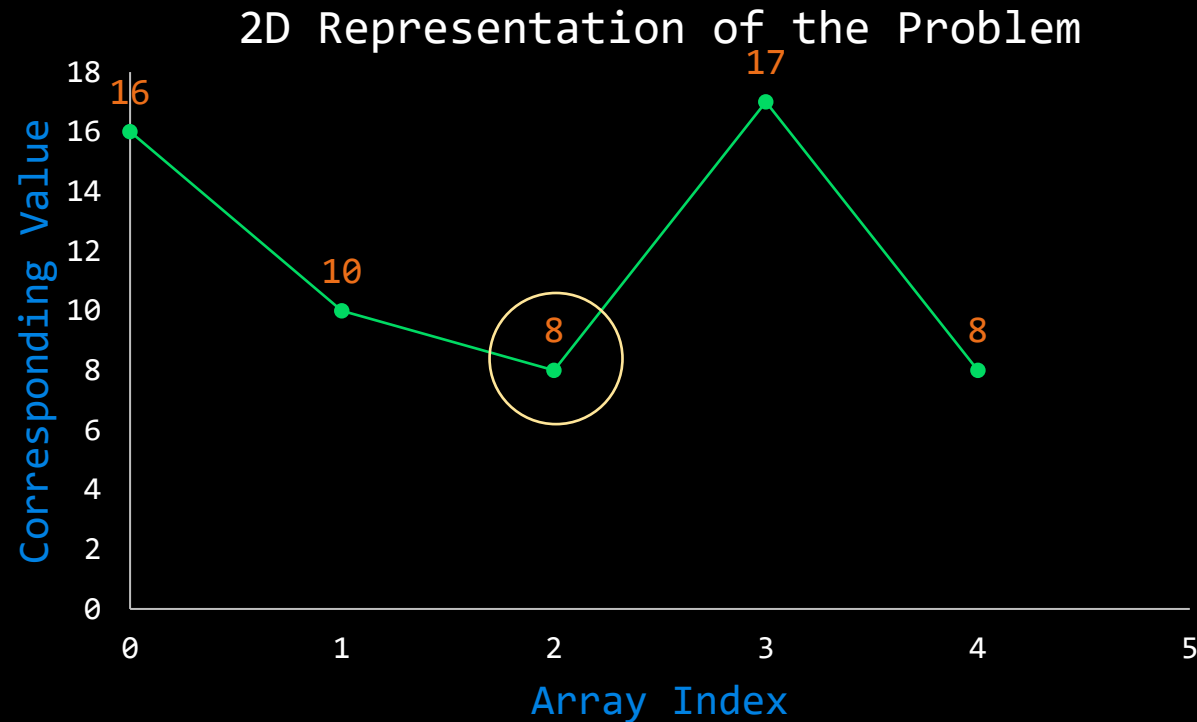
# Peak Finding: Case 4

|    |    |   |    |   |
|----|----|---|----|---|
| 16 | 10 | 8 | 17 | 8 |
|----|----|---|----|---|

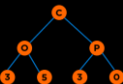


# Peak Finding: Case 4

|    |    |   |    |   |
|----|----|---|----|---|
| 16 | 10 | 8 | 17 | 8 |
|----|----|---|----|---|

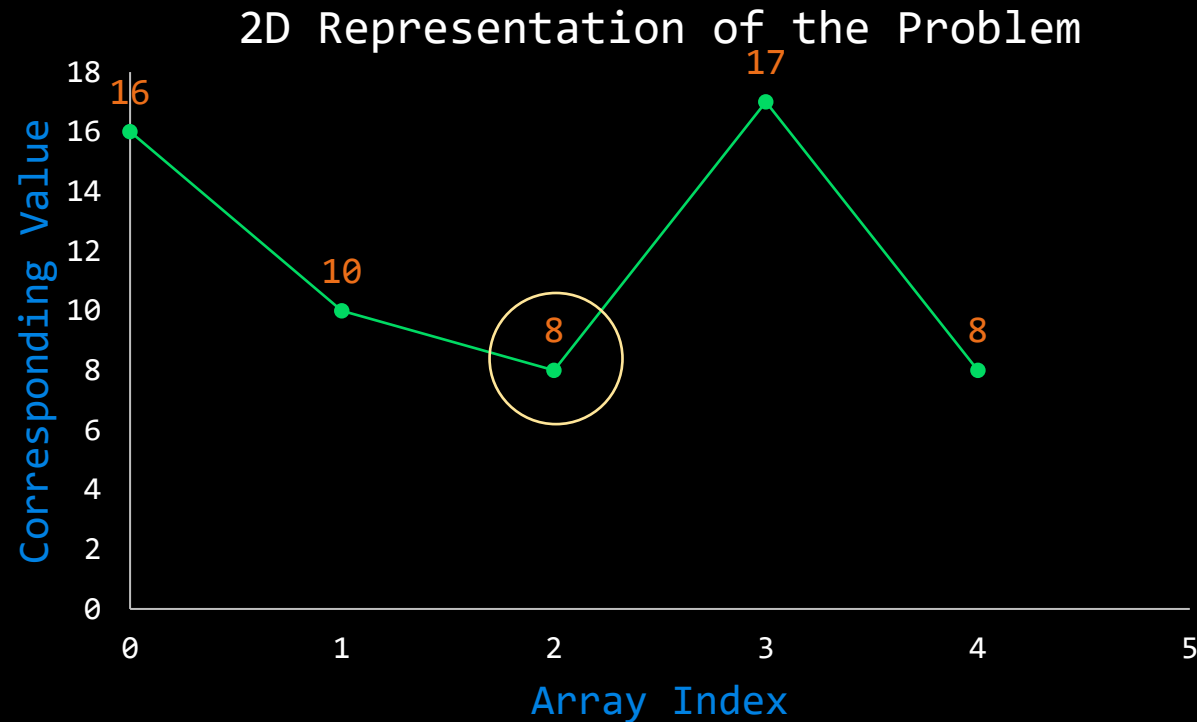


**Case 4: Central Element smaller than both adjacent elements:**

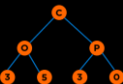


# Peak Finding: Case 4

|    |    |   |    |   |
|----|----|---|----|---|
| 16 | 10 | 8 | 17 | 8 |
|----|----|---|----|---|



**Case 4: Central Element smaller than both adjacent elements: Pick any side and keep going**



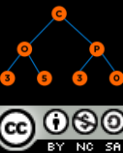
# Peak Finding

|    |    |   |    |   |
|----|----|---|----|---|
| 16 | 10 | 8 | 17 | 8 |
|----|----|---|----|---|

**Time Complexity:**

$O(\log_2 n)$  using the divide and conquer approach over  $O(n)$  using brute force algorithms

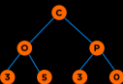
**One Solution:** <https://onlinegdb.com/YcfKNYnkT0>



# Binary Search

```
1.  int binarySearch(int arr[], int size, int target)
2.  {
3.      int start = 0, mid, end = size-1;
4.      while(start <= end)
5.      {
6.          mid = (start + end)/2;
7.          if(arr[mid] == target)
8.              return mid;
9.          else if(target > arr[mid])
10.             start = mid + 1;
11.          else
12.             end = mid - 1;
13.      }
14.      return -1;
15. }
```

<https://onlinegdb.com/S1GzxzljU>



# Questions